

Assignment 2 - Andrew Claxton

2026-06-04

```
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.5.3
```

```
## Loading required package: ggplot2
```

```
## Warning: package 'ggplot2' was built under R version 4.5.3
```

```
## Loading required package: lattice
```

```
library(class)
```

```
## Warning: package 'class' was built under R version 4.5.3
```

```
library(gmodels)
```

```
## Warning: package 'gmodels' was built under R version 4.5.3
```

```
Bankdf <- read.csv("C:\\Users\\User\\OneDrive\\Desktop\\Grad School_Business Analytics\\Fundamentals of
```

```
Bankdf$ID <- NULL
```

```
Bankdf$ZIP.Code <- NULL #drop ID and ZIP code from the data frame
```

```
Bankdf$Education <- factor(  
  Bankdf$Education,  
  levels = c(1, 2, 3),  
  labels = c("Undergrad", "Graduate", "Advanced")  
)
```

```
#convert education to a factor
```

```
dummy_model<-dummyVars(~Education, data=Bankdf) #model for creating dummy variables from Education column
```

```
edu.dummy<- predict(dummy_model, newdata = Bankdf) #make a new data frame with the dummy variables (in
```

```
Bankdf <- cbind(Bankdf, edu.dummy) #Add the edu.dummy data frame (two columns) to the master data frame
```

```
Bankdf$Education <- NULL #Drop the original categorical education field
```

Here we read our CSV, dropped irrelevant variables, converted Education to a factor, and then transformed the Education variable into dummy variables

```

set.seed(1) #seed for reproducibility
train.index <- createDataPartition(Bankdf$Personal.Loan, p = 0.6, list = FALSE) #stratified sampling ba
train.df <- Bankdf[train.index, ] #create training data frame from those indexes (randomly selected row
valid.df <- Bankdf[-train.index, ] #create the validation data frame from the indexes that were not in
norm.values <- preprocess(
  train.df[, -which(names(train.df) == "Personal.Loan")],
  method = c("center", "scale")
) #creating a model to normalize using Z-score. This normalization model is only training on the traini
train.norm.df <- predict(norm.values, train.df) #create a new data frame of normalized training data by
valid.norm.df <- predict(norm.values, valid.df) #same as above. New data frame created with normalized

```

We then create a data partition with 60% allocated for training and 40% allocated for validation. We normalized both training and validation data sets using z-score normalization (based on the training data)

```

target.customer <- data.frame( Age = 40, Experience = 10, Income = 84, Family = 2, CCAvg = 2, Education
target.customer.norm <- predict(norm.values, target.customer) #normalize the variables for the target c
knn.pred <- knn(
  train = train.norm.df[, -which(names(train.norm.df) == "Personal.Loan")],
  test = target.customer.norm,
  cl = train.norm.df$Personal.Loan,
  k = 1
)
# we train on our normalized test data frame using all columns except the variable we want to predict w
#our test data frame is the one we just created for the target customer. It is also normalized
# cl or class labels tells the model which column to look for the possible outcomes of the class of the
#k=1 means we are looking for the single nearest neighbor.
knn.pred #The knn() function will return a vector of predictions (0 or 1 dictated by the cl) against ou

```

```

## [1] 0
## Levels: 0 1

```

Question #1: The k-nn model with a k value of 1 predicted that the hypothetical customer's nearest neighbor in our test set was not likely to accept a loan offer. In other words, our model predicted that this customer will not convert from liability to asset.

```

accuracy <- rep(0, 14)
for (i in 1:14) {
  knn.pred <- knn(
    train = train.norm.df[, -which(names(train.norm.df) == "Personal.Loan")],
    test = valid.norm.df[, -which(names(valid.norm.df) == "Personal.Loan")],
    cl = train.norm.df$Personal.Loan,
    k = i
  )

  accuracy[i] <- mean(knn.pred == valid.norm.df$Personal.Loan)
}

accuracy.df <- data.frame(k = 1:14, accuracy = accuracy)

#Make a vector called accuracy with 14 values all equal to zero. Iterate through "i" in 1:14 trying a d
#we plug all of that into a new data frame called accuracy.df. We can plot it or just print it and look
accuracy.df

```

```
##      k accuracy
## 1    1    0.9630
## 2    2    0.9565
## 3    3    0.9640
## 4    4    0.9595
## 5    5    0.9605
## 6    6    0.9575
## 7    7    0.9580
## 8    8    0.9575
## 9    9    0.9535
## 10  10    0.9550
## 11  11    0.9495
## 12  12    0.9475
## 13  13    0.9500
## 14  14    0.9500
```

Question #2: We iterated through the model capturing accuracy values for each iteration as we changed the k value from 1 through 14. The choice of k that balances between overfitting and ignoring the predictor information is K=3.

```
knn.best <- knn(
  train = train.norm.df[, -which(names(train.norm.df) == "Personal.Loan")],
  test = valid.norm.df[, -which(names(valid.norm.df) == "Personal.Loan")],
  cl = train.norm.df$Personal.Loan,
  k = 3
)

CrossTable(
  x = valid.norm.df$Personal.Loan,
  y = knn.best,
  prop.chisq = FALSE
)
```

```
##
##
##      Cell Contents
## |-----|
## |                      N |
## |          N / Row Total |
## |          N / Col Total |
## |          N / Table Total |
## |-----|
##
##
## Total Observations in Table:  2000
##
##
##              | knn.best
## valid.norm.df$Personal.Loan |          0 |          1 | Row Total |
## -----|-----|-----|-----|
##              0 |      1786 |          9 |      1795 |
##              |      0.995 |      0.005 |      0.897 |
##              |      0.966 |      0.060 |          |
```

```
##          |      0.893 |      0.004 |          |
## -----|-----|-----|-----|
##          1 |        63 |       142 |       205 |
##          |      0.307 |      0.693 |      0.102 |
##          |      0.034 |      0.940 |          |
##          |      0.032 |      0.071 |          |
## -----|-----|-----|-----|
##          Column Total |       1849 |       151 |       2000 |
##          |      0.924 |      0.075 |          |
## -----|-----|-----|-----|
##
##
```

Question 3: This is the confusion matrix for the validation data that results from using the best k. Our model using the best k (3) resulted in 63 false negatives and 9 false positives out 2000 predictions.

```
best.knn.pred <- knn(
  train = train.norm.df[, -which(names(train.norm.df) == "Personal.Loan")],
  test = target.customer.norm,
  cl = train.norm.df$Personal.Loan,
  k = 3
)
best.knn.pred
```

```
## [1] 0
## Levels: 0 1
```

#Rerun the knn() function using our same normalized training data frame against the same target customer

Question #4: After running the k-nn model against the target customer but now with the best k value (3), the model still finds the target customer's three nearest neighbors to have a majority label of NOT likely to accept a loan. Therefore, the model predicts that the target customer will NOT accept a loan either. This is reasonable, as only 9.6% of customers in the original data set accepted the loan.

```
set.seed(1)
# 50% training
train.index <- createDataPartition(Bankdf$Personal.Loan, p = 0.5, list = FALSE)
train.df <- Bankdf[train.index, ]
temp.df <- Bankdf[-train.index, ]
# 30% validation (.5x.6 = .3)
valid.index <- createDataPartition(temp.df$Personal.Loan, p = 0.6, list = FALSE)
valid.df <- temp.df[valid.index, ]
test.df <- temp.df[-valid.index, ]
#First get the index of 50% of the data and set the train.df data frame to rows with that index. Take t
predictor.cols <- setdiff(names(train.df), "Personal.Loan")

norm.values <- preProcess(
  train.df[, predictor.cols],
  method = c("center", "scale")
)

train.norm.df <- predict(norm.values, train.df)
```

```

valid.norm.df <- predict(norm.values, valid.df)
test.norm.df <- predict(norm.values, test.df)
#normalize using only the training data set, then create new data frames for all three subsets
#Now run knn on all three sets
train.pred <- knn(
  train = train.norm.df[, -which(names(train.norm.df) == "Personal.Loan")],
  test = train.norm.df[, -which(names(train.norm.df) == "Personal.Loan")],
  cl = train.norm.df$Personal.Loan,
  k = 3
)
#knn from training to predict outcome of training.
valid.pred <- knn(
  train = train.norm.df[, -which(names(train.norm.df) == "Personal.Loan")],
  test = valid.norm.df[, -which(names(valid.norm.df) == "Personal.Loan")],
  cl = train.norm.df$Personal.Loan,
  k = 3
)
#knn from training to validation.
test.pred <- knn(
  train = train.norm.df[, -which(names(train.norm.df) == "Personal.Loan")],
  test = test.norm.df[, -which(names(test.norm.df) == "Personal.Loan")],
  cl = train.norm.df$Personal.Loan,
  k = 3
)
#knn from training to test

```

We then re-partitioned the data into a 50/30/20% split. After that, we normalized the data subsets. Then we ran k-nn functions for each of the test sets using the best k value (3)

```

# Training
CrossTable(train.norm.df$Personal.Loan, train.pred, prop.chisq = FALSE)

```

```

##
##
##      Cell Contents
## |-----|
## |                      N |
## |          N / Row Total |
## |          N / Col Total |
## |          N / Table Total |
## |-----|
##
##
## Total Observations in Table:  2500
##
##
##              | train.pred
## train.norm.df$Personal.Loan |          0 |          1 | Row Total |
## -----|-----|-----|-----|
##              0 |      2263 |          5 |      2268 |
##              |      0.998 |      0.002 |      0.907 |
##              |      0.977 |      0.027 |          |
##              |      0.905 |      0.002 |          |

```

```
## -----|-----|-----|-----|
##           1 |      54 |      178 |      232 |
##           |      0.233 |      0.767 |      0.093 |
##           |      0.023 |      0.973 |      |
##           |      0.022 |      0.071 |      |
## -----|-----|-----|-----|
##           Column Total |      2317 |      183 |      2500 |
##           |      0.927 |      0.073 |      |
## -----|-----|-----|-----|
##
##
```

Validation

```
CrossTable(valid.norm.df$Personal.Loan, valid.pred, prop.chisq = FALSE)
```

```
##
##
##   Cell Contents
## |-----|
## |               N |
## |   N / Row Total |
## |   N / Col Total |
## |   N / Table Total |
## |-----|
##
##
## Total Observations in Table:  1500
##
##
##               | valid.pred
## valid.norm.df$Personal.Loan |      0 |      1 | Row Total |
## -----|-----|-----|-----|
##           0 |      1358 |      6 |      1364 |
##           |      0.996 |      0.004 |      0.909 |
##           |      0.970 |      0.060 |      |
##           |      0.905 |      0.004 |      |
## -----|-----|-----|-----|
##           1 |      42 |      94 |      136 |
##           |      0.309 |      0.691 |      0.091 |
##           |      0.030 |      0.940 |      |
##           |      0.028 |      0.063 |      |
## -----|-----|-----|-----|
##           Column Total |      1400 |      100 |      1500 |
##           |      0.933 |      0.067 |      |
## -----|-----|-----|-----|
##
##
```

Test

```
CrossTable(test.norm.df$Personal.Loan, test.pred, prop.chisq = FALSE)
```

```
##
##
```

```

##      Cell Contents
## |-----|
## |                N |
## |      N / Row Total |
## |      N / Col Total |
## |      N / Table Total |
## |-----|
##
##
## Total Observations in Table:  1000
##
##
##               | test.pred
## test.norm.df$Personal.Loan |      0 |      1 | Row Total |
## -----|-----|-----|-----|
##               0 |      884 |      4 |      888 |
##               |      0.995 |      0.005 |      0.888 |
##               |      0.962 |      0.049 |      |
##               |      0.884 |      0.004 |      |
## -----|-----|-----|-----|
##               1 |      35 |      77 |      112 |
##               |      0.312 |      0.688 |      0.112 |
##               |      0.038 |      0.951 |      |
##               |      0.035 |      0.077 |      |
## -----|-----|-----|-----|
##               Column Total |      919 |      81 |      1000 |
##               |      0.919 |      0.081 |      |
## -----|-----|-----|-----|
##
##

```

Question #5: These are the three cross-tables for the training, validation, and test data. The model against the training data achieved accuracy of 97.6%. The model against the validation data achieved 96.8% accuracy. The model against the test data achieved 96.1% accuracy.

The accuracy from the training data was expected to be very high because this is the data we used as the reference points.

The accuracy from the validation data was expected and did indeed drop slightly because the model is trying to find nearest neighbors on unseen data. Accuracy was still high because although the partition is different, we have already performed tuning to find the best k

The accuracy on the test data dropped slightly further than the validation set. This is the generalized performance on unseen data. Inaccuracies are the result of overfitting on the training data.